

Retrievers

Retrievers are interfaces that return documents based on an unstructured query. They are more general than vector stores and do not need to store documents; they only need to retrieve them. Retrievers accept a string query as input and return a list of Document objects, which contain text content and metadata.

Types of Retrievers

LangChain supports various retrieval methods, including:

- *Vectorstore*: Uses vector embeddings for quick and easy retrieval. To use a vector store as a retriever, you can implement the `as_retriever` method.
- *ParentDocument*: Indexes chunks of documents but retrieves the whole document.
- *Multi Vector*: Creates multiple vectors for each document, useful for indexing summaries or questions.

Creating Custom Retrievers

To create a custom retriever, you extend the `BaseRetriever` class and implement the `_get_relevant_documents` method. This allows integration with databases or web requests. Once created, you can invoke the retriever using the `invoke` method.

Retriever Chains

Retriever chains are part of LangChain's workflow for integrating retrievers with LLMs. A retrieval chain typically involves:

1. *Retrieval*: Using a retriever to fetch relevant documents based on a query. You can create a retrieval chain using the `create_retrieval_chain` function.
2. *Combining Documents*: Optionally combining or summarizing the retrieved documents.
3. *LLM Inference*: Passing the combined documents to an LLM for generation or question answering.

The `create_retrieval_chain` function in LangChain facilitates this process by creating a chain that retrieves documents and then passes them to another chain for further processing.

Tools

Tools in LangChain are individual components that perform specific tasks, such as querying databases, interacting with APIs, or executing functions. They are essential for agents, which use these tools to interact with the environment.

Creating Tools

To create a tool from a function, you can use the `@tool` decorator. For creating a tool from a runnable, use `StructuredTool.from_function`. Once a tool is created, you can invoke it using the `invoke` method.

Examples of Tools

- *Wikipedia Tool*: Use `WikipediaRetriever`.
- *ArXiv Tool*: Use `ArxivAPIWrapper`.
- *Google Search Tool*: Use `GoogleSearch`.
- *Semantic Scholar API Tool*: Use `SemanticScholarAPI`.
- *Bing Search Tool*: Use `BingSearch`.
- *StackExchange Tool*: Use `StackExchange`.
- *Wolfram Alpha Tool*: Use `WolframAlpha`.

Agents

Agents are systems that use LLMs as reasoning engines to decide which actions to take and when. Unlike chains, which have a predefined sequence of actions, agents dynamically choose actions based on the output of the LLM.

Agents consist of:

- *LLM*: Acts as the reasoning engine.
- *Tools*: Perform specific tasks.
- *Memory*: Stores conversational history or intermediate results.

Building Agents

To build an agent, you define the LLM, tools, and memory components. LangChain provides functionalities like `create_react_agent`, `create_json_agent`, and `create_structured_chat_agent` to simplify this process.

Agent Executors

Agent Executors are the runtime components of agents. They execute the actions determined by the LLM and manage the interaction between the LLM, tools, and memory. Agent executors handle the flow of information and ensure that actions are executed in the correct order.

Key Features

- *Dynamic Decision-Making:* Agent executors use the LLM's output to decide which actions to take next.
- *Conversational Memory:* They maintain a memory of previous interactions, allowing for multi-turn conversations.
- *Tool Integration:* Agent executors manage the invocation of tools based on the LLM's decisions.

To execute agent actions, you can use the `stream` method of the `AgentExecutor`.